Scilight

Transactions on Graph Intelligence and Network Applications
https://www.sciltp.com/journals/tgina

Type of Paper (Article, Review, Communications, etc.)

NESS: Robust Graph Learning under Severe Missingness, and an Analysis of Self-Supervised and Auxiliary Objectives

Erfan Moshiri [1,†,*], Amin Beheshti [1,†], Melika Zare [1,†] and Parimehr Morassafar [1,†]

[1] School of Computing, Macquarie University, Sydney, Australia
* Correspondence: erfan.moshiri@hdr.mq.edu.au

† These authors contributed equally to this work.

How To Cite: Moshiri, E.; Beheshti, A.; Zareh, M.; Morassafar, P. NESS: Robust Graph Learning under Missingness, and an Analysis of Self-Supervised Objectives. *Transactions on Graph Intelligence and Network Applications* **2026**, *Volume*(Issue), Page Number. https://doi.org/10.xxxx/xxx

Received: day month year
Revised: day month year
Accepted: day month year
Published: day month year

Abstract: Graph neural networks assume every node carries an informative feature vector, yet in practice many nodes have no observed features at all, such as users without profiles, newly listed items, or failed sensors, and message passing degrades sharply when neighborhoods are empty. Recent works learn representations directly under missingness, using self-supervised multi-view methods that train the encoder on auxiliary tasks. But most of these do not study which combinations of auxiliary tasks and view-generation strategies benefit the downstream task most. They also evaluate on small binary datasets that do not represent most of the real-world information. We address both gaps. We introduce NESS, a graph neural network for learning under missing node features. It couples a Feature-Propagation prefill, which gives every missing node long-range structural signal, with a set of pluggable self-supervised objectives on a multi-view contrastive encoder. On top of it, we systematically study which objectives and view combinations help, at large scale. Evaluated using macro-F1 on downstream node classification, NESS improves over the strongest baseline on the large-scale ogbn-arxiv graph by up to 9.8% (relative) under high missingness. Our findings show that self-supervision generally helps, but the size of its benefit depends on the setting, such as the severity of missingness and the characteristics of the objective. Also, an objective helps most when its target is well aligned with the downstream task. These findings offer a principled basis for designing self-supervision under missing node features.

Keywords: missing node features; graph neural networks; self-supervised learning; node classification; feature propagation; contrastive learning;

1. Introduction

Graph neural networks (GNNs) have become the standard tool for learning on relational data, achieving strong results on node classification, link prediction, and recommendation [1, 2]. Their effectiveness rests on a quiet assumption: that every node carries an informative feature vector, which message passing then refines by aggregating information across edges. In practice this assumption often fails. Nodes join a network before a profile is filled in, newly listed items lack descriptions, sensors drop out, and privacy constraints withhold attributes. The result is a graph in which a substantial fraction of nodes have no observed features at all. Under such attribute-missing conditions, message passing degrades sharply: a node whose neighborhood is largely uninformative has little to aggregate, and the deficiency compounds with each additional missing hop. The problem is acute at scale: on a citation graph such as ogbn-arxiv (169K nodes), when, for example, 80% of nodes have no observed features, most local neighborhoods are almost entirely empty, and a standard GNN has little signal left to propagate.

We study this setting through the lens of the downstream task rather than reconstruction. Much prior work treats missing attributes as an imputation problem (fill in the missing features, then run a standard model) and measures success by how faithfully the raw features are recovered [3, 4]. Yet accurate reconstruction is neither necessary nor sufficient for an accurate decision boundary. What ultimately matters is whether the learned representations remain discriminative when features are scarce. We therefore adopt learning under missing node features: given a graph where whole feature vectors are absent, learn node representations that support accurate downstream node classification.

Approaches to missing node features fall into a few broad families, each with a characteristic limitation. Imputation-based methods reconstruct features before applying a GNN; propagation variants [5] are simple and scalable, while learned generative imputers [3, 4] are more expressive but rely on dense or per-node operations that do not scale. Imputation-free methods modify the aggregation to tolerate missing entries without filling them [6, 7]. Per-node parameterization assigns each missing node its own learnable embedding [8], which risks memorization when no supervision reaches those parameters. Most recently, self-supervised and contrastive methods learn missingness-robust representations from pretext tasks instead of relying on reconstruction quality [9, 10]. But these methods typically borrow a single auxiliary objective from the graph-SSL literature [11–13] without asking which objective is best suited to missingness. What is missing is a principled account of which self-supervised objectives help a GNN learn under missing features, and when.

This question is a natural continuation of our own prior work [14], which introduced a feature-based neighborhood self-supervised objective for graph attribute imputation, where it helped precisely because the goal was feature reconstruction and the objective supervised the same feature space. Here we ask a distinct question: does that auxiliary signal also improve the downstream classification boundary, and under what conditions? And how well does it apply to larger, dense-embedding graphs? Since reconstruction and classification are different objectives, the answer is not implied by the earlier result. We therefore treat self-supervision as an object of study rather than an assumption extending the earlier framework.

This paper substantially extends our conference work [14] along four axes. First, we shift the goal from feature imputation, evaluated by reconstruction quality, to downstream node classification under missingness. Second, we replace the earlier single feature-based objective with NESS, a more mature model that couples a Feature-Propagation prefill and a multi-view contrastive encoder with a family of pluggable objectives, and that attains stronger results. Third, we contribute the systematic study that motivates this paper: rather than assuming one objective, we analyze which self-supervised objectives help under missingness and why. Fourth, we scale the evaluation beyond the small binary-attribute graphs of the conference version to a large dense-embedding graph (ogbn-arxiv), the regime in which the effect is most pronounced.

We introduce NESS, a GNN for learning under missing node features that couples (i) a Feature-Propagation prefill that gives every missing node long-range structural signal before training, and (ii) a set of self-supervised objectives layered on a well-configured encoder. NESS outperforms imputation-based, imputation-free, and per-node-parameter baselines on downstream classification, with the advantage most pronounced under severe missingness and at a scale where generative imputation is infeasible.

Beyond the method, our central contribution is a systematic study of self-supervised objectives for learning under missingness. We find that self-supervision generally helps, but the size of its benefit depends on the setting. Two factors modulate it. First, the severity of missingness: self-supervision contributes most when the encoder is information-starved, and its advantage grows as features become scarcer. Second, the nature of the features: neighborhood-centric objectives are more effective on dense, continuous features, but less on sparse binary bag-of-words attributes. We further find that the benefit varies with the objective itself, and analyze which properties make an objective effective (Section 6.4). A label-aware objective is the most reliable, and it transfers as an enhancement

to other backbones. This also motivates our focus on large dense-embedding graphs, the regime prior small-scale binary benchmarks least represent.

Our contributions are as follows:

- We introduce NESS, a graph neural network that couples a Feature-Propagation prefill with pluggable self-supervised objectives, and show that it outperforms imputation-based, imputation-free, and per-node-parameter baselines, with the largest advantage under severe missingness and at scale.
- We provide an empirical study of when and why self-supervision helps under missingness, showing that its benefit generally holds but is modulated by the severity of missingness, the type of features, and the characteristics of the objective, which we analyze rather than assume.
- We identify two reliable objectives for our setting, a label-aware neighbor-histogram and a structure-based objective, and show that they transfer as an enhancement to other GNN backbones.

In the following sections we review related work on graph learning under weak information (Section 2), discuss the different self-supervised objectives used in graph learning (Section 3), introduce our method (Section 4), and conduct comprehensive experiments to address our research questions (Section 5).

2. Related Work

Graph learning problems can be distinguished by what information is unavailable and by the extent of the missing information. For node features, attribute-incomplete graphs contain missing feature entries, while attribute-missing graphs contain nodes whose entire feature vectors are unavailable. Some studies consider several forms of incomplete information together. For example, D²PT jointly handles incomplete features, incomplete structure, and limited labels [15], while T2-GNN addresses incomplete feature entries and incomplete structure using separate feature and structure teachers to reduce interference between the two sources of information [16]. Other studies focus mainly on label scarcity, where only a small number of node labels are available. Graph Structure Learning methods, such as IDGL and Pro-GNN, learn or correct graph topology while assuming that node features are observed [17, 18]. In this work, we focus on node classification when the entire feature vectors of some nodes are missing, while the graph structure is fully observed and labels are available for the downstream task. In the first part, we discuss handling missing information in graphs through imputation and learning under missingness. We then discuss graph learning with self-supervised methods.

2.1. Graph Learning Under Missing Information

Graphs that contain missing information can be handled in different ways. Imputation-based methods first estimate the missing feature values and then use the completed feature matrix for a downstream graph-learning task. Learning-under-missingness, or imputation-free, methods instead learn reliable representations directly from the incomplete graph for use in downstream tasks.

2.1.1. Statistical and Propagation-Based Imputation

Statistical and propagation-based imputation methods first estimate missing node features and then apply a standard GNN to the completed feature matrix. Simple methods such as K-nearest neighbour (KNN) imputation and neighbour aggregation estimate missing features using similar nodes or directly connected neighbours [19]. These methods are computationally efficient and scalable, but they use fixed local rules and do not learn how features should be imputed [4]. Feature Propagation (FP) extends neighbour-based aggregation through a parameter-free diffusion process [5]. It repeatedly propagates features through the normalized graph adjacency while resetting observed entries to their original values, allowing information to travel across multiple hops, and aims to recover feature information that supports downstream classification rather than faithfully reconstructing the original features. It scales to large graphs and performs well under high missing rates, although its effectiveness depends on graph homophily and decreases when connected nodes are dissimilar. Overall, these methods provide simple and scalable imputations, but their fixed propagation rules cannot learn task-specific feature reconstruction.

2.1.2. Learned and Generative Imputation

Learned imputation methods use trainable models to estimate missing features from observed attributes, graph structure, or both. SAT addresses attribute-missing graphs by mapping structural and attribute information into a shared latent space [3]. It uses the structural representation of a node to generate its missing feature vector and is trained through reconstruction and adversarial distribution-matching objectives. SVGA also considers

the attribute-missing setting but introduces a structured variational model with a graph-based Gaussian Markov Random Field prior [4]. It uses graph regularization to model dependencies between nodes and can optionally use observed labels as an additional training signal. Both methods directly optimize feature reconstruction and then evaluate whether the estimated features support downstream node classification. SAT was evaluated on graphs with up to approximately 20,000 nodes, while SVGA extended the evaluation to OGBN-Arxiv with approximately 169,000 nodes. However, both methods assume undirected homogeneous graphs and do not address entry-wise attribute-incomplete settings.

A broader body of work provides methodological background but does not address attribute-missing graphs directly. GAIN and GINN impute entry-wise missing values in tabular data, the latter through a similarity graph and a graph convolutional denoising autoencoder [20, 21], while GC-MC and IGMC apply graph autoencoders to matrix completion in recommender systems [22, 23]. These methods complete tabular entries or missing edges rather than fully missing node feature vectors, and so are not direct solutions to our setting.

ITR initializes each attribute-missing node using its structure-based representation and then refines the initialized representations through an affinity matrix [24]. The affinity matrix combines the observed graph structure with similarities calculated from the updated node representations. Reliable representations of attribute-observed nodes are restored during each refinement step to reduce the spread of imputation errors. RITR extends this approach to graphs that contain both partially missing feature entries and fully missing node feature vectors [25]. For the pure attribute-missing setting, its refinement process follows the original ITR design. Both methods are transductive and rely on graph-wide affinity operations, while RITR reports quadratic complexity and requires subgraph sampling on large datasets.

Amer also learns missing attributes jointly with graph representations, but it does not assign an independent parameter vector to each missing node [26]. Instead, a shared generator produces missing attributes from random noise, and structural reconstruction, mutual-information, and adversarial objectives guide the generated values. Amer is therefore more accurately treated as a generative and structure-guided method than as a per-node parameterization method.

Overall, learned imputation methods can model more complex relationships than fixed propagation rules, but they scale less well and are largely transductive, and their experiments show that improved feature reconstruction does not always improve downstream prediction. We therefore evaluate missing-feature methods through downstream node classification and on larger graphs, and favour methods that learn from shared structural patterns rather than node-specific quantities.

2.2. Imputation-Free and Architectural Methods

Imputation-free methods modify the GNN architecture to process missing features directly, without first producing a completed feature matrix. GCNMF models missing feature values using a Gaussian mixture model and computes the expected activation of the first GCN layer under this distribution [7]. The remaining layers follow the standard GCN architecture, and the feature model and GCN parameters are trained jointly using the downstream task loss. This design allows missing-feature handling to be guided directly by node classification or link prediction rather than by a separate reconstruction objective. However, GCNMF was evaluated only on small- and medium-sized graphs, with the largest reported dataset containing approximately 13,700 nodes.

PaGCN follows a deterministic architectural approach based on partial aggregation [6]. Its aggregation operator uses a feature mask so that only observed entries from neighbouring nodes contribute to each feature channel. The operator also adjusts the normalization according to the number of available contributors and reduces to standard graph convolution when all features are observed. PaGCN introduces no additional learnable parameters beyond the GNN and is trained end-to-end using the node classification loss. It supports both entry-wise and whole-vector missingness and was evaluated on graphs with up to approximately 169,000 nodes. However, its experiments are limited to node classification, and some computationally heavier baselines were not evaluated on the largest datasets.

We mention PCFI here for contrast, as it is closely related in spirit but follows an impute-first design rather than an imputation-free architecture [27]. It estimates missing features through confidence-weighted diffusion before applying a downstream GNN, performing well under high missing rates but relying on feature homophily, and its results again show that better downstream accuracy does not necessarily correspond to more accurate feature recovery.

Overall, GCNMF and PaGCN handle missingness within the GNN forward pass and are optimized through supervised downstream objectives, without adding a self-supervised auxiliary signal. PaGCN provides the broader large-graph evaluation among these architecture-integrated methods. PCFI offers an efficient propagation-based alternative, but it belongs more directly to the imputation-based family.

2.3. Self-Supervised and Multi-View Graph Learning

Self-supervised learning (SSL) methods help the model learn auxiliary signals that improve embedding quality. SSL is also an appealing alternative to imputation on attribute-missing graphs. Instead of filling in absent inputs, an SSL method learns representations from the unlabeled structure and is judged by how well those representations support downstream tasks. Our work belongs to this line, so we review it in detail.

The next two subsections discuss general SSL graph-learning methods that operate on fully observed graphs. We then discuss methods that adapt SSL to learning under missing information.

2.3.1. Contrastive and Multi-View Self-Supervision

The dominant approach learns node representations by making different views of a graph agree, usually framed as maximizing a lower bound on their mutual information. Methods in this family differ in the granularity of the agreement and in how the views are produced. Deep Graph Infomax [28] maximizes the mutual information between patch-level node representations and a graph-level summary, avoids random-walk objectives, and works in both transductive and inductive settings. GMI [29] measures mutual information between a node's hidden representation and its neighborhood over both features and topology, which ties the signal to local structure. MVGRL [13] contrasts an adjacency view against a graph-diffusion view. Its authors report that adding more than two views or contrasting multi-scale encodings does not help, and that the strongest results come from contrasting first-order neighbors with the diffusion view.

A large group of methods build views by perturbing the input at random. GRACE [30] corrupts structure and attributes to form two views and contrasts them node by node. GCA [31] makes the corruption adaptive, perturbing less important edges and features more aggressively according to centrality and attribute-importance scores. GraphCL [32] catalogs four families of graph augmentation and studies how they combine, and later work automates the choice because it otherwise has to be retuned for each dataset. Handcrafted augmentation can change what a graph means, so a parallel line reduces or removes it. AFGRL [33] forms a second view without augmentation by finding nodes that share local structure and global semantics with the target. MA-GCL [34] perturbs the view encoders' architectures, through asymmetric, random, and shuffling tricks, rather than the graph itself. A final group drops negative pairs and decorrelates the embedding dimensions in the spirit of Barlow Twins [12], whose graph counterpart is DCLN [35].

Taken together, these methods show that the objective and the view both matter, and that generic augmentations are a brittle design axis. All of them, however, are built and tested on fully observed graphs. Their views come from augmentations that are not designed for absent features, and none of them asks which objective still carries information once whole attribute vectors are gone. That is the question we take up.

2.3.2. Masked and Reconstruction-Based Self-Supervision

A second family of pretext tasks corrupts part of the input and trains the model to restore it, an idea imported from masked modeling in language and vision. These methods differ in what they mask. GraphMAE [11] masks node features and reconstructs them with a scaled cosine error and a re-masking step, and shows that a plain autoencoder can then match or exceed contrastive baselines. MaskGAE [36] masks structure instead, treating masked edge prediction as the pretext and reconstructing the removed edges from the visible ones, with theory that links the objective to contrastive learning. RARE [37] masks and predicts in the latent space rather than the input space for added robustness. These objectives are attractive because they need no negative sampling and carry over a recipe that has worked well elsewhere.

Masking, though, is a pretext applied to data that is otherwise complete. When attributes are genuinely missing, the feature-reconstruction signal largely overlaps with what neighborhood aggregation already recovers, so it adds relatively little to what the encoder computes on its own. Our experiments bear this out. A pure feature-reconstruction objective tends to plateau early and yields only a limited downstream gain under missingness. Faithful reconstruction and a useful representation are not the same goal in this regime.

2.4. Self-Supervision for Learning Under Missingness

This group learns representations from graphs with missing information using self-supervision, and these methods are the closest to ours. AmGCL [9] works in two stages. A Dirichlet-energy-minimization precoder writes information into the missing entries, and a self-supervised contrastive module then learns representations by maximizing a lower bound on the mutual information of the latent code, together with a structure-attribute energy-based reconstruction. AmGCL commits to a single contrastive design, does not analyze which objective or view

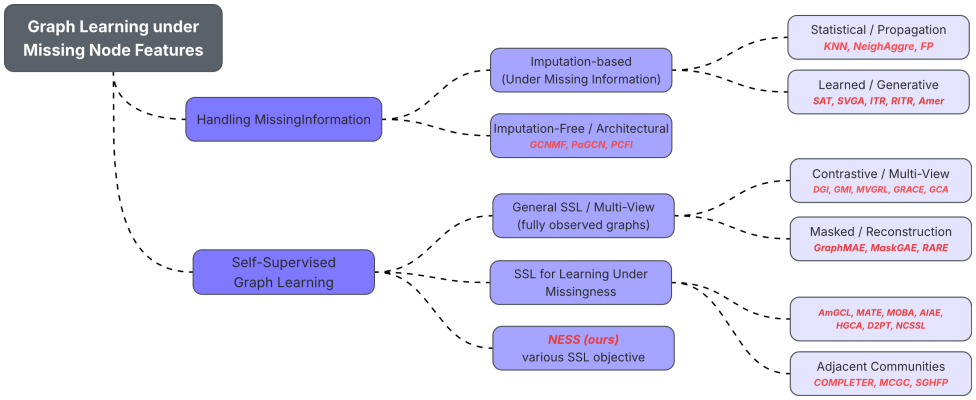

Figure 1. Taxonomy of methods reviewed in Section 2.

produces its gains, and is evaluated only at small scale. MATE [8] takes a multi-view route, building complementary views of the graph and enforcing agreement across them for imputation. Because supervision never reaches its per-node augmented quantities directly, such designs are prone to memorization, and shallow encoders can be starved of signal when missingness is high. MATE also learns node-specific values or representations within a fixed graph and does not generalize to unseen attribute-missing nodes. MOBA [10] shares the multi-view philosophy but sets out to fix these weaknesses, using a collaborative strategy that cuts redundant information between views while keeping them consistent. Its view design is still fixed in advance, it does not explain why a given view or objective works, and it is not demonstrated at large scale. AIAE [38] is autoencoder-based and reconstruction-centric, framed around imputation quality rather than downstream accuracy under severe missingness. Outside the homogeneous setting, HGCA [39] joins attribute completion and representation learning for heterogeneous graphs through an unsupervised contrastive objective, completing missing attributes at a fine granularity with an augmented network. Our own conference work [40] introduced a neighborhood-centric self-supervised formulation, which the present study extends.

D²PT leans on graph topology when node features provide limited information, and considers graph learning with incomplete features, incomplete structure, and limited labels at the same time [15]. It first applies long-range PageRank-based propagation to spread the available feature information across the graph. It then constructs a global k-nearest-neighbour graph from the propagated features and aligns the representations learned from the original and global graph channels. The model is trained using classification losses and a prototype-alignment objective. D²PT does not explicitly reconstruct missing features, and its experiments consider entry-wise feature missingness together with missing edges and limited labels. It therefore addresses a broader weak-information setting rather than features-only attribute-missing learning.

The pattern across these methods is consistent. Each fixes one objective and one view and tunes it, and none investigates which self-supervised objective and which view help under missingness, or why. Their results are also confined to small graphs and usually a single missing rate, so behavior across the missingness range and at scale is left unexamined.

2.4.1. Adjacent Communities

Two nearby literatures combine missingness with contrastive learning under a different task or connectivity model. Incomplete multi-view clustering pairs absent information with contrastive objectives but targets clustering rather than node classification. COMPLETER [41] handles the views of an incomplete multi-view dataset with a contrastive-prediction objective, and MCGC [42] learns a consensus graph with a contrastive regularizer because the observed graph may be noisy or incomplete. On the higher-order side, SGHFP [43] propagates features over hypergraphs to cope with missing attributes. We note these links but do not treat them as central, since they differ from our problem in task or in graph model.

2.5. Positioning of Our Work

Figure 1 illustrates an breakdown summary of related work. Seen together, the prior work leaves a clear opening. Imputation methods optimize reconstruction rather than downstream utility, and the learned variants do not scale or generalize to unseen missing nodes; imputation-free architectural methods add no self-supervisory signal. General graph SSL is powerful but untested for missing features, and its own literature shows view construction to be a brittle, handcrafted axis. The SSL methods built for attribute-missing graphs each fix one objective and one view without asking which choices matter, and report results only at small scale and a single missing rate.

Our work responds to this gap. We evaluate representations by downstream node-classification accuracy under missingness, at substantially larger scale and across a range of missing rates, and conduct a systematic study of which self-supervised objectives and views help. We find that self-supervision generally helps, but its benefit is governed by the severity of missingness and by how class-relevant and non-trivial the objective's target is. The effect is not specific to our encoder: the objective transfers to other backbones, indicating that the principle, not the architecture, drives the gains.

3. Self-Supervised Objectives

A self-supervised (SSL) objective trains the encoder to predict a target derived from the graph itself, without human labels. Under missing features it is appealing for a specific reason: the target is computed from information that survives missingness (graph structure, observed neighbors, or observed labels), so the objective can shape the representations of nodes whose own features are absent. By forcing the encoder to solve an auxiliary prediction task, SSL can inject signal that pure classification, trained only on observable nodes, does not reach.

Many SSL objectives exist, each defined by a different prediction target. We organize this design space along four axes.

(1) Type (primary axis).

Our primary categorization is by the source of the prediction target. *Feature-related* objectives predict statistics of a node's neighbors' features, such as the neighborhood mean or spread, or reconstruct masked feature values; they assume the features themselves carry the signal. *Structural* objectives predict topological quantities computed purely from the graph, such as reachability between nodes, hop distances, landmark distances, or centrality, and thus remain defined even when no features are observed. *Label-related* objectives predict a target built from the observed training labels, for instance the class distribution of a node's neighbors; they are semi-supervised, using labels available at training time but requiring none at inference. *Contrastive* objectives do not predict a fixed target at all: they encode two views of the graph and encourage agreement (or reduced redundancy) between the resulting representations. Strictly speaking, the label-related objectives are semi-supervised rather than purely self-supervised, since they use observed training labels; we include them in this taxonomy of auxiliary objectives because they share the same pluggable role and are compared on equal footing with the others.

(2) Locality.

The receptive field of the target. A *local* target depends only on a node's immediate neighborhood, its adjacent nodes and their features or labels, and is therefore recoverable by a few rounds of message passing. A *global* target depends on a node's position in the graph as a whole, such as its distance to distant landmark nodes or its centrality, and captures long-range structure that local aggregation does not directly expose.

(3) Static vs. stochastic.

Whether the target is fixed for the whole of training or refreshed each step. A *static* target is computed once and reused every epoch, so the objective repeatedly presents the same supervision. A *stochastic* target is resampled every step, or every few steps, for example by drawing fresh random walks, sampling new node pairs, or reselecting landmark nodes, so the objective presents a continually changing supervision signal drawn from the same underlying distribution. A static target is cheap and stable, whereas a stochastic one presents a fresh problem each step and resists memorization.

(4) Input-varying vs. fixed-input.

What the objective takes as input. A *fixed-input* (per-node) objective maps a single node's representation to a target that is a fixed function of that node; the encoder sees the same input–target pair on every visit, so once it fits each node the objective stops providing new information. An *input-varying* (relational) objective takes a freshly sampled set of nodes and predicts a relation among them, for example whether two sampled nodes are within a few hops or which of two nodes is closer to a landmark, so each step poses a genuinely new problem rather than repeating a memorized one. Note this is distinct from axis (3): a target can be stochastically resampled yet still be per-node (fixed-input), or relational.

Table 1 places representative objectives on these axes, grouped by type.

Table 1. Taxonomy of self-supervised objectives, grouped by type. "Target" names the predicted quantity; "Input" is per-node (fixed) or relational (rel., varying). PE denotes positional encoding and betw. betweenness centrality.

Type	Objective	Target	Locality	Static/Stoch.	Input
Feature-related	centroid	neighbor mean	local	static	per-node
	stats	neighbor std	local	static	per-node
	recon	masked node features	local	stochastic	per-node
	denoising	denoised features	local	stochastic	per-node
Structural	path	multi-hop reachability	local	stochastic	relational
	triplet	distance ranking	local	stochastic	relational
	common-neighbor	pairwise overlap score	local	stochastic	relational
	random-walk PE	return probabilities	local – mid	static/stoch.	per-node
	anchor	distance to landmarks	global	stochastic	per-node / rel.
	anchorcls	landmark-distance bucket	global	stochastic	per-node
	shortest-path	pairwise hop distance	global	stochastic	relational
	PageRank	global importance	global	static	per-node
	k-core / betw.	density / role	global	static	per-node
Label-related	hist	neighbor label histogram	local	static	per-node
	pseudo-label	propagated pseudo-labels	local	stochastic	per-node
	label-smooth	label smoothness	local	static	per-node
Contrastive	Barlow-Twins	view decorrelation	global	stochastic	relational
	InfoNCE / GRACE	node view agreement	local	stochastic	relational
	prototype-InfoNCE	class-prototype agree.	global	stochastic	relational
	DGI / GMI	node–summary MI	global / local	stochastic	relational
	BYOL / AFGRL	bootstrap agreement	local	stochastic	relational

Discussion on difficulty.

Beyond these four axes, objectives also differ in how hard their target is to fit. An objective whose target has little variance across nodes, such as the mean of neighbor features that message passing tends to produce anyway, is easy: the loss falls to a small value within a few epochs and then stays flat. An objective whose target has high entropy or depends on a freshly sampled set of nodes each step, such as predicting a label distribution or a relation between sampled nodes, is hard: the loss keeps decreasing over training as the encoder is repeatedly asked new questions. Difficulty therefore follows from the axes above: static, low-variance, feature-space targets tend to be easy, whereas high-entropy or input-varying targets tend to be hard. In this work we examine whether this difficulty relates to the quality of the learned embeddings.

In this work we implement and evaluate a representative subset spanning all four types: *feature-related* (centroid, stats, recon); *structural* (path, triplet, anchor, and its classification variant anchorcls); *label-related* (hist); and *contrastive* (Barlow-Twins, InfoNCE, and prototype-InfoNCE). The remaining objectives in Table 1 are included to situate these choices within the broader design space. Formal definitions of the evaluated objectives are given in Appendix 8.

4. Method

We present **NESS**, a framework for learning under missing node features. NESS is deliberately modular: a feature-propagation prefill and a message-passing encoder produce node representations, onto which a set of *pluggable* self-supervised objectives is layered. This modularity is intentional. Our study of which objectives help under missingness (Section 5) is enabled by treating the auxiliary objective as an interchangeable component rather than a fixed part of the architecture.

4.1. Problem Formulation and Notation

Let $G = (V, E, \mathbf{X})$ be an undirected attributed graph with $n = |V|$ nodes, edge set E, adjacency matrix $\mathbf{A} \in \{0,1\}^{n\times n}$, and feature matrix $\mathbf{X} \in \mathbb{R}^{n\times d}$. Each node v_i has a label $y_i \in \{1, \ldots, C\}$. We denote by $\tilde{\mathbf{A}} = \mathbf{D}^{-1/2}(\mathbf{A} + \mathbf{I})\mathbf{D}^{-1/2}$ the symmetrically normalized adjacency with self-loops, where $\mathbf{D}$ is the degree matrix.

Under *attribute-missing* conditions, the node set partitions into an **observable** set V_o and a **missing** set V_m, with $V_o \cup V_m = V$ and $V_o \cap V_m = \emptyset$. For a missing node $v_i \in V_m$, the entire feature vector is unobserved; we

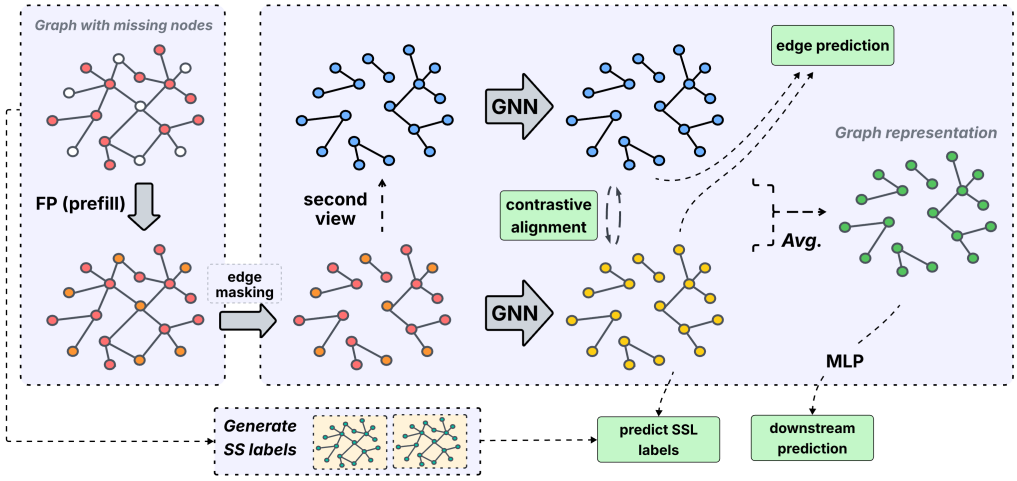

Figure 2. The pipeline of NESS. We first apply Feature Propagation to prefill missing nodes. We then optimize the contrastive and self-supervised objectives. Green boxes indicate loss objectives.

represent the observed input by zero-filling,

$$\bar{\mathbf{x}}_i = \begin{cases} \mathbf{x}_i, & v_i \in V_o, \\ \mathbf{0}, & v_i \in V_m, \end{cases} \tag{1}$$

giving the masked feature matrix $\bar{\mathbf{X}}$. Following a completely random missing pattern, V_m is sampled uniformly at a rate $\rho = |V_m|/n$. The goal is *not* to reconstruct $\mathbf{X}$ but to learn node representations $\mathbf{Z} \in \mathbb{R}^{n \times h}$ that support accurate classification of the missing nodes V_m, measured by macro-F1.

4.2. Overview

NESS comprises four stages (Figure 2): (1) a *feature-propagation prefill* that fills missing rows of $\bar{\mathbf{X}}$ with long-range structural signal; (2) a message-passing *encoder* that maps the prefilled features to representations $\mathbf{Z}$; (3) a *classification head* trained on observable labels; and (4) a set of *self-supervised objectives* that shape $\mathbf{Z}$. The total training loss combines the supervised and self-supervised terms. We describe each stage below.

4.3. Feature-Propagation Prefill

A shallow message-passing encoder starves when a node's neighborhood is largely empty, which is precisely the regime induced by high missingness. To supply missing nodes with signal before encoding, we prefill $\bar{\mathbf{X}}$ using Feature Propagation [5]: features are diffused over the graph while observed rows are reset to their true values at every step. Starting from $\mathbf{H}^{(0)} = \bar{\mathbf{X}}$,

$$\mathbf{H}^{(t+1)} = \tilde{\mathbf{A}}\,\mathbf{H}^{(t)}, \qquad \mathbf{H}^{(t+1)}_{i,:} = \mathbf{x}_i \;\; \forall\, v_i \in V_o, \tag{2}$$

iterated for T steps (we use $T = 40$). The prefilled matrix $\hat{\mathbf{X}} = \mathbf{H}^{(T)}$ replaces $\bar{\mathbf{X}}$ as the encoder input. The reset in (2) enforces the observed values as boundary conditions, so the diffusion solves a Dirichlet energy minimization that propagates observed features into missing regions. Prefill is a one-time preprocessing step of cost $O(|E|\,d\,T)$.

4.4. Encoder and Views

The encoder f_θ is an L-layer GraphSAGE [1] network. At layer ℓ, node v_i updates its representation by combining a transform of its own state with an aggregate of its neighbors $\mathcal{N}(i)$,

$$\mathbf{h}_i^{(\ell)} = \sigma\Big(\mathbf{W}_{\text{self}}^{(\ell)}\mathbf{h}_i^{(\ell-1)} + \mathbf{W}_{\text{nbr}}^{(\ell)}\,\text{mean}_{j\in\mathcal{N}(i)}\,\mathbf{h}_j^{(\ell-1)}\Big), \tag{3}$$

with $\mathbf{h}_i^{(0)} = \hat{\mathbf{x}}_i$ and σ a nonlinearity. The separate self-transform $\mathbf{W}_{\text{self}}$ preserves each node's own (prefilled) signal through depth, which we find important under missingness (Section 5).

To support view-based objectives, we form two stochastic views by independent random edge-masking of G,

encode each, and fuse them into the final representation,

$$\mathbf{Z}^{(1)} = f_\theta(\hat{\mathbf{X}}, G^{(1)}), \quad \mathbf{Z}^{(2)} = f_\theta(\hat{\mathbf{X}}, G^{(2)}), \quad \mathbf{Z} = \tfrac{1}{2}\big(\mathbf{Z}^{(1)} + \mathbf{Z}^{(2)}\big), \tag{4}$$

where $G^{(1)}, G^{(2)}$ are edge-masked variants of G, and we denote by E_m the edges removed to form $G^{(1)}$, which serve as targets for the edge-reconstruction objective below. The fused $\mathbf{Z}$ is used by the classification head and all objectives. The view-generation method is itself a modular choice; we adopt independent edge-masking, which we found effective in our setting.

4.5. Learning Objectives

Classification: A head g_ϕ maps representations to class logits, trained with cross-entropy on *observable* nodes only (no label leakage to V_m),

$$\mathcal{L}_{\text{cls}} = \frac{1}{|V_o|} \sum_{v_i \in V_o} \text{CE}\big(g_\phi(\mathbf{z}_i),\, y_i\big). \tag{5}$$

Edge reconstruction: A structural objective encourages $\mathbf{Z}$ to preserve connectivity. The positives are the edges held out when forming view $G^{(1)}$ in (4); using an edge decoder ψ, these are contrasted against sampled negatives E^-, a set of randomly drawn non-adjacent node pairs,

$$\mathcal{L}_{\text{edge}} = -\sum_{(i,j)\in E_m} \log \sigma\big(\psi(\mathbf{z}_i, \mathbf{z}_j)\big) - \sum_{(i,j)\in E^-} \log\big(1 - \sigma(\psi(\mathbf{z}_i, \mathbf{z}_j))\big), \tag{6}$$

where E_m is the set of masked edges.

Contrastive (view agreement): We align the two views with a redundancy-reduction (Barlow-Twins [12]) objective on their cross-correlation matrix $\mathcal{C}$, computed over the batch dimension of the projected views,

$$\mathcal{L}_{\text{con}} = \sum_k (1 - \mathcal{C}_{kk})^2 + \lambda \sum_k \sum_{k' \neq k} \mathcal{C}_{kk'}^2. \tag{7}$$

Self-supervised auxiliary objective: The distinctive component of NESS is a pluggable self-supervised objective $\mathcal{L}_{\text{ssl}}$ that shapes $\mathbf{Z}$ using structure- or label-derived targets requiring no missing-node features. Rather than committing to a single objective, we treat $\mathcal{L}_{\text{ssl}}$ as an interchangeable component and study a family of candidates spanning distinct design axes: *feature-neighborhood* (predict the neighborhood mean or spread), *feature-reconstruction* (masked-feature recovery), *structural* (multi-hop reachability, distance ranking, anchor-distance to landmarks), and *label-based* (neighbor label-histogram, semi-supervised). Each is defined formally in Appendix 8; their comparative analysis, which objectives help and when, is the subject of Section 5.

Our reported model instantiates $\mathcal{L}_{\text{ssl}}$ with two objectives. The label-histogram objective predicts, for each observable node, the class distribution $\mathbf{p}_i$ of its observable neighbors, using a head h_{hist} trained with a KL divergence,

$$\mathcal{L}_{\text{hist}} = \frac{1}{|V_o|} \sum_{v_i \in V_o} \text{KL}\big(\mathbf{p}_i \,\|\, \text{softmax}(h_{\text{hist}}(\mathbf{z}_i))\big). \tag{8}$$

The label-histogram target is computed exclusively from the labels of observable (training) nodes; validation and test labels never enter any target, so the objective introduces no leakage of evaluation labels. The target for node i is an aggregate of its graph neighbors' labels: the adjacency carries no self-loops, so a node's own label does not appear in its one-hop histogram. Because the objective reuses only labels already available to the supervised loss, it adds no information beyond the standard training signal, and is best understood as a semi-supervised auxiliary target rather than a source of leakage.

The multi-hop reachability (path) objective samples random walks to obtain node pairs (s, e) reachable within a few hops as positives and distant pairs (s, e^-) as negatives, and scores them with a head h_{path},

$$\mathcal{L}_{\text{path}} = \tfrac{1}{2}\Big[\text{BCE}\big(h_{\text{path}}(\mathbf{z}_s \odot \mathbf{z}_e),\, 1\big) + \text{BCE}\big(h_{\text{path}}(\mathbf{z}_s \odot \mathbf{z}_{e^-}),\, 0\big)\Big]. \tag{9}$$

Here $\mathcal{L}_{\text{ssl}} = \mathcal{L}_{\text{hist}} + \mathcal{L}_{\text{path}}$.

Total objective: The training loss is a weighted combination,

$$\mathcal{L} = w_{\text{cls}}\,\mathcal{L}_{\text{cls}} + w_{\text{edge}}\,\mathcal{L}_{\text{edge}} + w_{\text{con}}\,\mathcal{L}_{\text{con}} + w_{\text{ssl}}\,\mathcal{L}_{\text{ssl}}, \tag{10}$$

with nonnegative weights $w_{\cdot}$. Setting $w_{\text{ssl}} = 0$ recovers a strong no-SSL baseline used throughout our analysis; it still retains the classification, edge-reconstruction, and contrastive terms, and drops only the auxiliary self-supervised objective.

4.6. Training

For large graphs, we partition G into clusters following [44] and train full-batch within each cluster; small graphs are trained full-batch directly. Self-supervised targets that depend on non-local information (e.g. landmark distances, or label histograms aggregated over a multi-hop neighborhood) are computed once on the full graph and indexed per cluster. The per-epoch cost is dominated by the encoder forward/backward pass, $O(|E|\, h + n\, h^2)$ per cluster, matching standard message-passing GNNs. Full hyperparameter and optimization settings are given in Section 6.

5. Experiments and Results

Our experiments serve two goals. We first establish that NESS is an effective model for learning under missing node features: it outperforms baselines spanning every category of the problem (Section 6.1), and its advantage widens as features become scarcer (Section 6.2). We then turn to the study that motivates the paper: not merely whether self-supervision helps, but under which conditions it does and what explains its effectiveness. We show that its benefit is conditional: only some objectives help (Section 6.3), for reasons visible in their mechanism and training dynamics (Section 6.4); the benefit emerges primarily under severe missingness (Section 6.5) and on dense rather than sparse binary features (Section 6.6). Finally, we justify NESS's design choices and establish its transferability and scalability (Sections 6.7 and 6.9).

6. Experimental Setup

Datasets.

We evaluate on five node-classification benchmarks. Four are small but conventional benchmarks in the graph-missingness literature, citation and co-purchase graphs with sparse binary bag-of-words attributes: Cora, CiteSeer [45], Amazon-Computers (AMAC), and Amazon-Photo (AMAP) [46]. The fifth, ogbn-arxiv [47], is a large graph with dense, continuous features (169K nodes, 128-dimensional embeddings). We center our study on ogbn-arxiv as a larger, more realistic benchmark whose dense continuous features and scale expose challenges that small binary graphs do not, making it better suited to studying self-supervision under missingness. Their statistics are summarized in Table 2. This spread lets us study behavior across both feature regimes (sparse binary vs. dense continuous) and scales.

Missingness protocol.

We simulate attribute-missing conditions by selecting a fraction ρ of nodes uniformly at random and removing their entire feature vector (zero-filled). Nodes labels are divided into training, validation, and test splits. Message passing operates over the full graph throughout, so missing nodes still receive representations via propagation from their observed neighbors; only their labels and input features are withheld. We use $\rho = 0.4$ for the main comparison against other benchmarks, 0.8 for the rest of the experiments, and sweep $\rho \in \{0.2, 0.4, 0.6, 0.8, 0.9, 0.95\}$ for the robustness analysis.

Methods compared.

We compare NESS against baselines spanning every category of the missing-feature landscape. **NeighAggre** and **KNN** are non-parametric imputers that fill a missing node from its neighbors or nearest observed nodes [19]. **Feature Propagation** (FP) is a scalable propagation-based imputer that diffuses observed features over the graph [5]. **PaGCN** is an imputation-free architecture that aggregates only observed feature entries [6]. **MATE** is a per-node method that learns a separate embedding for each missing node [8]. **GraphSAGE** is a standard zero-filled GNN baseline [1].

We report macro-F1 as the primary metric, along with accuracy on the masked test nodes. All results are averaged over three random seeds.

All learnable methods are trained under an identical budget and protocol: the same number of epochs (800, with early stopping on validation macro-F1, patience of 20 evaluation intervals), the same optimizer (Adam), the same per-dataset learning rate, the same hidden width (128), and the same missingness splits and seeds. Comparisons therefore reflect the methods themselves rather than differences in tuning. The learning rate is 10^{-3} for the large

Table 2. Dataset statistics. ogbn-arxiv is our primary large graph dataset.

Dataset	Nodes	Edges	Features	Classes	Feature type
Cora	2,708	5,429	1,433	7	binary BoW
CiteSeer	3,327	4,732	3,703	6	binary BoW
AMAP	7,650	119,081	745	8	binary BoW
AMAC	13,752	245,861	767	10	binary BoW
ogbn-arxiv	169,343	1,166,243	128	40	dense embedding

Table 3. Main comparison under random missingness at rate 0.4. Each cell reports macro-F1 / accuracy on masked nodes, averaged over three seeds. Best macro-F1 per dataset in **bold**.

	Cora		CiteSeer		AMAC		AMAP		ogbn-arxiv	
Method	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc
NeighAggre	0.804	0.799	0.658	0.675	0.848	0.865	0.868	0.888	0.325	0.570
KNN	0.811	0.812	0.689	0.720	0.835	0.847	0.841	0.870	0.335	0.578
GraphSAGE	0.837	0.840	0.646	0.681	0.865	0.879	0.900	0.916	0.372	0.639
FP	**0.864**	**0.862**	0.720	0.744	0.893	0.899	0.887	0.907	0.439	0.673
PaGCN	0.846	0.847	0.666	0.686	0.904	0.909	**0.918**	**0.925**	0.383	0.631
MATE	0.826	0.827	0.678	0.705	0.906	0.912	0.912	0.924	0.404	0.651
NESS	0.842	0.836	**0.735**	**0.751**	**0.911**	**0.924**	0.908	0.916	**0.454**	**0.676**

and denser graphs (ogbn-arxiv, AMAC), where a higher rate is unstable, and 10^{-2} for the remaining small graphs, applied uniformly to every method on a given dataset.

NESS configuration.

Unless otherwise noted, NESS uses a three-layer GraphSAGE encoder of hidden width 128, Feature-Propagation prefill (40 iterations), two independently edge-masked views fused by averaging with a Barlow-Twins contrastive term, and the label-histogram and multi-hop reachability self-supervised objectives. Dropout is 0.3. The loss weights place higher emphasis on the classification term (Section 5 reports the selection). Large graphs are trained with cluster-based batching; small graphs are trained full-batch. Self-supervised targets that depend on global structure are computed once on the full graph and indexed per cluster. The full hyperparameter configuration is discussed in Appendix 8.

6.1. Main Comparison

We first compare NESS against baselines under random missingness at rate 0.4. Table 3 reports test macro-F1 and accuracy on the masked nodes, averaged over three seeds.

On ogbn-arxiv, the large dense-embedding graph that is the primary target of this work, NESS attains the highest performance, reaching 0.454 macro-F1 compared to 0.439 for the strongest baseline (FP) and 0.404 for MATE, and the highest accuracy (0.676). NESS also performs favorably on the small binary graphs, achieving the best macro-F1 on CiteSeer (0.735) and AMAC (0.911) and results within a narrow margin of the best on Cora and AMAP. On these small, sparse benchmarks the strongest methods (FP, PaGCN, MATE) perform comparably and the differences are small; a more pronounced separation emerges on the large dense graph, consistent with the regime analysis in Section 6.6 and with the widening advantage under increasing missingness reported in Section 6.2.

6.2. Robustness to Missingness Rate

To assess how each method behaves as features become increasingly scarce, we vary the missing rate ρ from 0.2 to 0.9 on ogbn-arxiv and measure test macro-F1 on the masked nodes. Table 4 reports the results, averaged over three seeds, for NESS and the strongest baselines from Section 6.1.

NESS attains the highest macro-F1 at every rate except 0.6, where MATE is marginally higher (0.458 versus 0.449). More importantly, its performance is the most stable across the range, declining only from 0.449 at $\rho = 0.2$ to 0.415 at $\rho = 0.9$. The baselines degrade substantially more over the same interval: FP falls from 0.442 to 0.354, PaGCN from 0.402 to 0.256, and KNN from 0.345 to 0.258. Consequently, the margin in favor of NESS widens as missingness increases, and it is largest at the most severe rate (0.415 versus 0.378 for MATE and 0.354 for FP at $\rho = 0.9$). This indicates that the advantage of NESS is most pronounced precisely in the regime where the problem

Table 4. Macro-F1 on ogbn-arxiv as a function of the missing rate ρ. The best result at each rate is shown in bold.

Missing rate (ρ)	KNN	FP	PaGCN	MATE	NESS
0.2	0.345	0.442	0.402	0.387	**0.449**
0.4	0.335	0.437	0.384	0.394	**0.454**
0.6	0.317	0.437	0.355	**0.458**	0.449
0.8	0.291	0.415	0.302	0.412	**0.444**
0.9	0.258	0.354	0.256	0.378	**0.415**

Table 5. Objective ablation on ogbn-arxiv at 80% missingness. Δ is relative to the no-SSL baseline.

Configuration	Test F1	Δ vs. no-SSL
hist + path (default)	**0.447**	+0.022
hist	0.444	+0.020
recon	0.441	+0.017
path	0.432	+0.007
centroid	0.431	+0.007
anchor	0.429	+0.004
triplet	0.426	+0.002
no-SSL	0.424	(baseline)
stats	0.421	−0.003
single-view	0.416	−0.009

is hardest and where features are least available.

6.3. Which Objectives Help

We next study the self-supervised objectives themselves. Holding the encoder and all other components fixed, we train NESS with one objective at a time and with the combined default (hist + path), and compare against a no-SSL baseline (classification only) and a single-view variant that removes the second view and the contrastive term. Table 5 reports test macro-F1 on ogbn-arxiv at 80% missingness.

Self-supervision helps in most configurations: all but one objective improve over the no-SSL baseline, and the combined set (hist + path) is best, improving macro-F1 by 0.022. The label-aware neighbor-histogram (hist) is the strongest single objective (+0.020), followed by feature reconstruction (+0.017); the structural and feature-statistic objectives give smaller positive lifts, and only the neighbor-spread objective (stats) is marginally negative. The usefulness of feature reconstruction is consistent with our prior work on feature-based self-supervision for imputation [40]. Removing the second view and contrastive term (single-view) yields the weakest configuration, below even no-SSL, indicating that the two-view contrastive structure itself contributes. We note that these gains are modest in absolute terms: the bulk of NESS's performance comes from the proper choice of prefill, encoder, and contrastive views, with self-supervision providing a consistent additional lift. The objectives thus differ substantially in how much they help; Section 6.4 identifies the property that explains this spread.

6.4. Why Objectives Help

The objective ablation in Section 6.3 shows that objectives differ widely in how much they help. Here we investigate one property associated with these gains: how class-relevant the objective's target is. For each objective with a per-node target, we fit a logistic-regression probe, a simple classifier trained to predict a node's class from the objective's target alone, and measure its macro-F1 as a proxy for the class-relevance of that target. We then relate it to the objective's downstream lift from Section 6.3. Relational objectives (path, triplet), whose target is a relation between sampled nodes rather than a per-node vector, are excluded from this analysis.

Figure 4 plots the two quantities. The more class-relevant an objective's target, the more it improves downstream classification: the neighbor label-histogram (hist) has both the most class-relevant target and the largest lift, whereas the neighbor-spread target (stats) is the least class-relevant and is the only objective that does not help. Feature reconstruction (recon) also provides a useful signal, consistent with our prior work on feature-based self-supervision for imputation [40]. This explains why the label-aware histogram is the most reliable objective, and why we pair it with a structural objective in NESS.

The training dynamics corroborate this picture. Figure 3 plots the per-objective self-supervised loss over

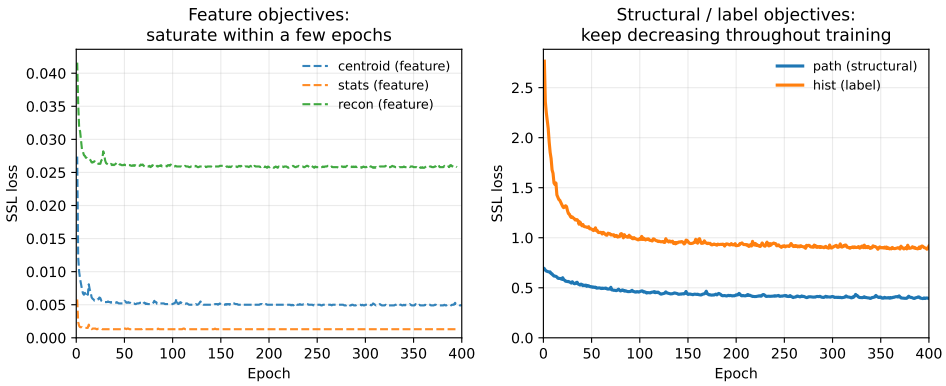

Figure 3. Per-objective self-supervised loss over training (each curve is from a run using that objective alone). Left: feature objectives saturate within a few epochs. Right: structural and label objectives keep decreasing throughout training.

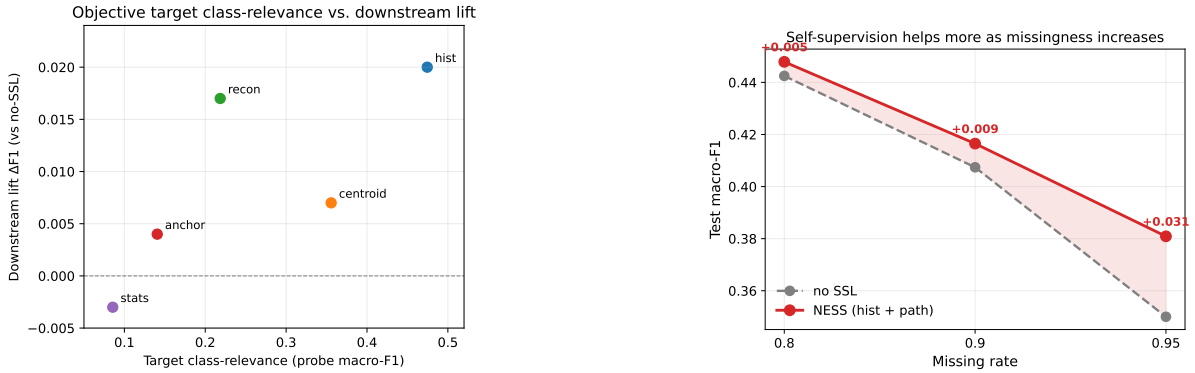

Figure 4. (Left) Target class-relevance versus downstream lift for per-node-target objectives. Each objective's target is probed for class information (x-axis); the more class-relevant the target, the larger the downstream improvement (y-axis).

Figure 5. (Right) Test macro-F1 on ogbn-arxiv as the missing rate increases. Both NESS (hist + path) and the no-SSL baseline decline, but the self-supervised model declines more slowly.

training. Feature-space objectives with low-variance targets fall to a small value within a few epochs and then remain flat, providing little further gradient, while the label and structural objectives begin at a higher loss and continue to decrease throughout training, supplying informative supervision for longer. An objective is therefore useful when its target is both class-relevant and non-trivial to fit.

Finally, we verified that a class-irrelevant objective cannot be made useful by tuning: for a structural distance objective whose target is only weakly class-relevant, neither increasing its loss weight, adding a projection head between the encoder and the objective, nor scheduling the objective before classification changed the outcome. The determining factor is the class-relevance of the target, not the weight or attachment of the objective.

6.5. Self-Supervision under Severe Missingness

The benefit of self-supervision depends on how much the encoder is starved of features. To make this precise, we compare NESS (with its hist and path objectives) against the no-SSL baseline as the missing rate increases from 0.8 to 0.95 on ogbn-arxiv. Figure 5 shows the result. Both configurations degrade as features become scarcer, but the no-SSL baseline degrades faster, so the advantage of self-supervision widens: the gain over no-SSL grows from $+0.005$ at $\rho = 0.8$ to $+0.031$ at $\rho = 0.95$. When observed features are abundant the encoder already recovers most of the signal and self-supervision adds little; as the graph becomes information-starved, the self-supervised objectives contribute an increasingly large share of the usable signal. This is the regime in which self-supervision is most valuable.

6.6. The Feature Regime

The benefit of self-supervision also depends on the type of node features. Neighborhood-centric objectives are built from aggregates of neighbor features, which are informative only when the features themselves are informative

Table 6. Self-supervision on sparse binary graphs at 80% missingness, with the dense-embedding result (ogbn-arxiv) for reference. On binary attributes the change over no-SSL is within noise; on dense features it is clearly positive.

Dataset	Features	no-SSL	Δ with SSL
Cora	binary	0.798	0.000
CiteSeer	binary	0.595	$+0.002$
AMAC	binary	0.863	-0.005
AMAP	binary	0.876	$+0.000$
ogbn-arxiv	dense	0.424	$+0.020$

under aggregation. To test this, we apply the two label- and structure-based objectives (hist and path) to the four small graphs, whose attributes are sparse binary bag-of-words vectors, and compare the resulting lift to that observed on the dense-embedding graph ogbn-arxiv. Table 6 reports test macro-F1 at 80% missingness.

On the sparse binary graphs the objectives provide essentially no benefit: the change relative to the no-SSL baseline is within ± 0.01 on all four datasets, with no consistent direction. This contrasts sharply with the dense-embedding graph, where the same objectives improve macro-F1 by 0.020. The explanation is mechanistic: neighborhood-centric targets require averaging neighbor features, which yields a meaningful, predictable signal for dense continuous attributes but is close to noise for sparse binary indicators. Neighborhood self-supervision is therefore effective in the dense-feature regime, which is both the practically prevalent one and the setting on which we focus.

6.7. Design Choices and Sensitivity

We now justify NESS's design choices and show that its performance is not brittle to hyperparameters. All studies are on ogbn-arxiv at 80% missingness.

Design choices:

Figure 6 compares the two principal architectural decisions, each varied in isolation. For the encoder, GraphSAGE outperforms GAT and GCN; its separate self-transformation preserves each node's prefilled signal through depth, unlike GCN-style aggregation that mixes a node's own signal with its neighbors' in a single term. For the prefill, Feature Propagation clearly outperforms a single round of neighbor-mean imputation and zero-filling, confirming that long-range diffusion recovers signal that a single aggregation step or no imputation cannot. (The two panels are separate ablations and share only the metric axis; absolute values are not directly comparable across them.)

Component ablation:

Table 7 removes NESS's learning components one at a time. Removing the self-supervised objectives (no-SSL) and, more so, the entire two-view contrastive structure (single-view) both reduce performance, and removing the Feature-Propagation prefill (zero-fill) is the most damaging. Each component contributes, and the prefill is the single most important.

Hyperparameter sensitivity:

Figure 7 varies four hyperparameters one at a time around the chosen configuration; all values are selected on validation performance. Performance is stable, and the chosen settings lie within a stable region: a deeper (three-layer) encoder and lower dropout (0.3) are preferable, and the learning rate is stable around 10^{-3}. Up-weighting the classification term improves results and then plateaus; we set its weight to 3, a stable point on this plateau rather than the extreme, to avoid over-emphasizing the supervised term at the expense of the auxiliary signal that matters at higher missingness.

6.8. Transferability of the Objective

To show that an effective SSL objective reflects a general principle rather than a property of our architecture, we test whether it improves other backbones as well. We add the label-histogram objective to two baselines, GraphSAGE and PaGCN, as an auxiliary loss and measure the change in test macro-F1 on ogbn-arxiv at 80% missingness. Table 8 shows that both backbones improve when the objective is added, indicating that the benefit

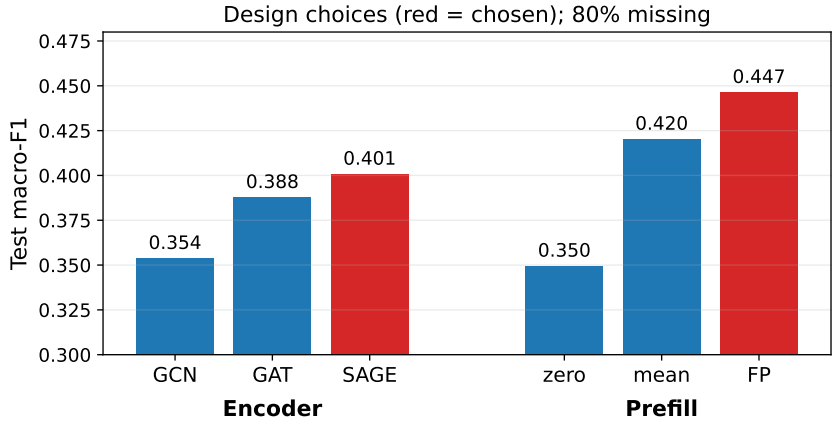

Figure 6. Design choices (chosen option in red), on ogbn-arxiv at 80% missingness. GraphSAGE is the best encoder and Feature Propagation the best prefill. The two panels are independent ablations.

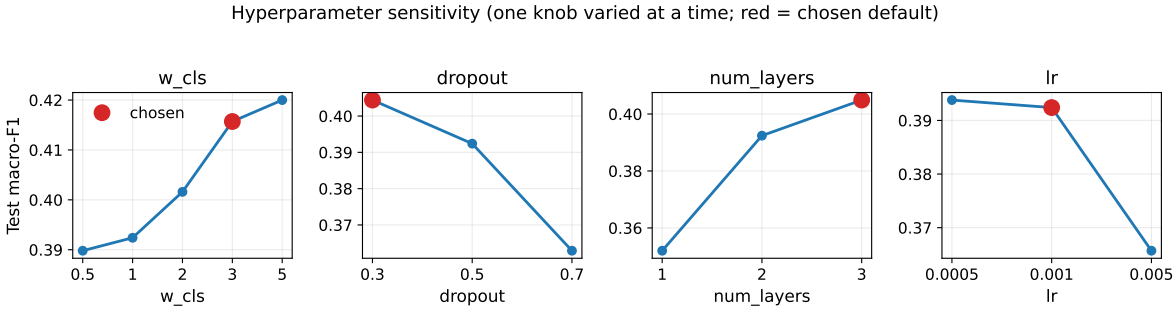

Figure 7. Hyperparameter sensitivity: each knob is varied with the others fixed; the chosen default is marked in red. Performance is stable, and the selected settings lie within a stable region.

transfers across architectures and is driven by the objective rather than by NESS specifically. The gains are modest, consistent with the objective supplying a semi-supervised signal that is most valuable where the backbone is otherwise weak.

6.9. Scalability

Finally, we confirm that NESS is practical at scale. Figure 8 plots test macro-F1 against peak GPU memory for all methods on ogbn-arxiv, with bubble size indicating training time. NESS attains the highest macro-F1 at a peak memory of 1.3 GB, comparable to the per-node baseline (MATE) and well within a single GPU; its self-supervised objectives and prefill add only a small memory overhead relative to a standard clustered GNN. Its training time is higher than the simpler baselines (larger bubble), reflecting the auxiliary objectives computed each epoch, but remains on the order of minutes on a single GPU.

7. Discussion

The two factors that govern the benefit of self-supervision, the severity of missingness and the characteristics of the objective, admit a single interpretation: an objective helps to the extent that it supplies usable signal beyond what message passing already recovers. When observed features are abundant the encoder is not starved, so even a well-chosen objective adds little; when the target is class-irrelevant or trivially predictable, an objective adds little at any level of missingness. The largest gains therefore arise when both conditions align, that is, when a class-relevant, non-trivial objective is trained under severe missingness. This reading unifies the missingness-severity and objective-characteristic results under one mechanism rather than treating them as separate effects.

For practitioners, this suggests a concrete criterion for adding self-supervision under missing features: prefer objectives whose targets are class-relevant and not trivially predictable, expect the benefit to be largest under severe missingness and on dense continuous features, and expect little from self-supervision when features are abundant or sparse and binary. This stands in contrast to the common practice of importing an auxiliary objective from the graph self-supervision literature by analogy, without regard to whether its target carries downstream-relevant signal in the missing-feature regime. It also reconciles our findings with our earlier work on feature-based self-supervision

Table 7. Component ablation on ogbn-arxiv at 80% missingness (test macro-F1, three seeds). Removing any learning component reduces performance; the prefill matters most.

Configuration	Test F1
NESS (full)	**0.447**
without SSL objectives	0.424
without two-view contrastive	0.416
without FP prefill (zero-fill)	0.350

Table 8. Adding the label-histogram objective to other backbones (on ogbn-arxiv, 80% missingness, macro-F1).

Backbone	without	with hist	Δ
GraphSAGE	0.217	0.229	+0.012
PaGCN	0.300	0.318	+0.018

for imputation: feature reconstruction remains useful, but because faithful reconstruction is neither necessary nor sufficient for an accurate decision boundary, a label-aware objective is the more reliable choice when the goal is classification rather than recovery.

It is also worth being precise about the source of NESS's overall strength. At moderate missingness, much of its performance derives from the encoder itself, namely the Feature-Propagation prefill together with a well-configured GraphSAGE backbone, while the self-supervised objectives act as a setting-dependent enhancement whose contribution grows as missingness becomes severe. Rather than the advantage resting on self-supervision alone, it is the combination that makes the model strong: the encoder and the auxiliary objectives are complementary, and together they add value across a range of settings and missing rates.

Limitations and future work: Several limitations bound these conclusions and point to natural directions for future work. Our study considers missingness sampled uniformly at random, so whether the same criterion holds under structured or informative missingness remains open. We evaluate only on node classification, and extending the analysis to other downstream tasks, such as link prediction and regression, would test the generality of the class-relevance criterion. Our probe of target class-relevance is diagnostic after training rather than predictive before it; a measure of an objective's usefulness that can be computed in advance would turn the analysis into a design tool. Finally, the conclusion that the self-supervised task is ineffective on sparse binary attributes is drawn from small graphs, and a large sparse-attribute benchmark would strengthen it.

8. Conclusion

We introduced NESS, a graph learning model under missing node features that couples a Feature-Propagation prefill with auxiliary self-supervised objectives on a well-configured encoder, outperforming imputation-based, imputation-free, and per-node-parameter baselines on downstream node classification, most notably under severe missingness and at scale. We further present an analysis of when self-supervision is beneficial in this setting. Self-supervision is generally helpful, yet the magnitude of its benefit is governed by the severity of missingness and by the characteristics of the objective, with more class-relevant and less trivially predictable targets yielding larger gains. The label-aware neighbor-histogram is the most reliable objective and transfers to other backbones. Together, these results give a principled basis for choosing self-supervision under missing node features.

Author Contributions

For research articles with several authors, a short paragraph specifying their individual contributions must be provided. The following statements should be used, "X.X.: conceptualization, methodology, software; X.X.: data curation, writing—original draft preparation; X.X.: visualization, investigation; X.X.: supervision; X.X.: software, validation; X.X.: writing—reviewing and editing. All authors have read and agreed to the published version of the manuscript". Please turn to the CRediT taxonomy for the term explanation. Authorship must be limited to those who have contributed substantially to the work reported.

Funding

This research was funded by the Centre for Applied Artificial Intelligence at Macquarie University and Domain Holdings Pty Ltd. **(grant number needed?)**

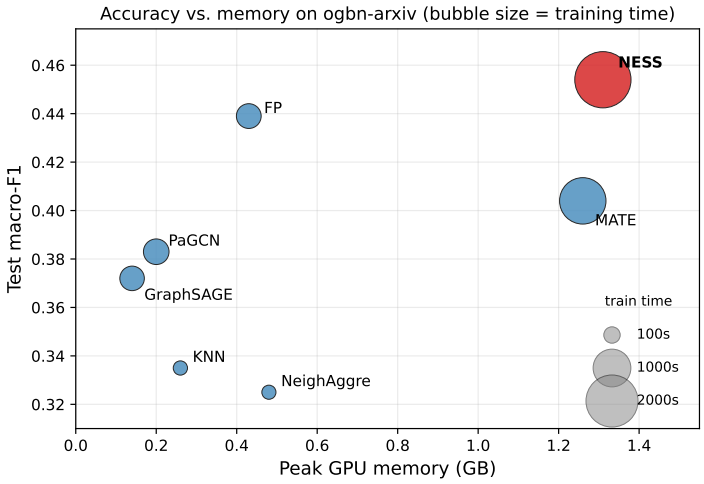

Figure 8. Macro-F1 versus peak GPU memory on ogbn-arxiv at 80% missingness; bubble size is training time. NESS (red) attains the highest macro-F1 at memory comparable to other GNNs, confirming it scales with manageable memory usage.

Data Availability Statement

The code for NESS is available at https://github.com/erfanmoshiri/NESS. All datasets used in this study are publicly available. The Cora and CiteSeer citation graphs are provided by the Planetoid collection [45], and the Amazon co-purchase graphs AMAC (Amazon-Computers) and AMAP (Amazon-Photo) are provided by [46]; both are accessible through the PyTorch Geometric library (https://pytorch-geometric.readthedocs.io). The ogbn-arxiv graph is provided by the Open Graph Benchmark [47] and is available at https://ogb.stanford.edu.

Acknowledgments

We acknowledge the Centre for Applied Artificial Intelligence at Macquarie University and Domain Holdings Pty Ltd for funding this research. We also acknowledge the use of AI-assisted language tools for editorial refinement during manuscript preparation.

Conflicts of Interest

The authors declare no conflict of interest.

Use of AI and AI-Assisted Technologies

We also acknowledge the use of AI-assisted tools during the preparation of this manuscript, including for editorial refinement and the generation of selected draft content such as statements and figures. All AI-assisted outputs were reviewed, verified, and approved by the authors, who take full responsibility for the final content of the manuscript.

Appendix A. Formal Definitions of Evaluated Objectives

This appendix gives the precise definition of each self-supervised objective evaluated in Section 6.3. All objectives operate on the encoder output $\mathbf{Z}$ (Section 4.4) and use only information that survives missingness, namely graph structure, observed neighbor features, or observed training labels. We write $\mathcal{N}_k(i)$ for the k-hop neighborhood of node i, V_o for the observable set, and $h_{(\cdot)}$ for a lightweight per-objective prediction head applied to $\mathbf{Z}$. Unless noted, k matches the value used in training.

Feature-related objectives.

These predict per-node targets derived from observable neighbor features and are trained with a mean-squared error. The *centroid* objective predicts the mean of a node's observable neighbors' features (for nodes with a non-empty observable neighborhood),

$$\mathbf{t}_i^{\text{cen}} = \frac{1}{|\mathcal{N}_k(i) \cap V_o|} \sum_{j \in \mathcal{N}_k(i) \cap V_o} \mathbf{x}_j, \qquad \mathcal{L}_{\text{cen}} = \frac{1}{n} \sum_i \left\| h_{\text{cen}}(\mathbf{z}_i) - \mathbf{t}_i^{\text{cen}} \right\|_2^2. \tag{11}$$

The *stats* objective is identical but predicts the per-dimension standard deviation of the same neighborhood, $\mathbf{t}_i^{\text{std}} = \text{std}_{j \in \mathcal{N}_k(i) \cap V_o} \mathbf{x}_j$. The *recon* objective is a masked-feature self-reconstruction: at each step half of the observable nodes are held out, their features zeroed, and their true features regressed from the re-encoded representations,

$$\mathcal{L}_{\text{rec}} = \frac{1}{|H|} \sum_{i \in H} \left\| h_{\text{rec}}(\mathbf{z}_i) - \mathbf{x}_i \right\|_2^2, \tag{12}$$

where $H \subset V_o$ is the freshly sampled held-out set.

Structural objectives.

These predict topological targets computed purely from the graph and are defined even when no features are observed. The *path* objective is a stochastic multi-hop link prediction. A fresh batch of random walks of length $\ell = 2$ yields positive pairs (s, e) with e reachable within ℓ hops of s; negatives (s, e^-) draw e^- from the set of nodes at least 3 hops from s. Since $\ell < 3$, the positive and negative sets do not overlap. With a scoring head h_{path} on the elementwise product of endpoints,

$$\mathcal{L}_{\text{path}} = \tfrac{1}{2} \Big[\text{BCE}\big(h_{\text{path}}(\mathbf{z}_s \odot \mathbf{z}_e),\, 1\big) + \text{BCE}\big(h_{\text{path}}(\mathbf{z}_s \odot \mathbf{z}_{e^-}),\, 0\big) \Big]. \tag{13}$$

The *triplet* objective ranks distances in embedding space: for an anchor a, a near node p (within ℓ hops) and a far node q (≥ 3 hops), it enforces a margin,

$$\mathcal{L}_{\text{tri}} = \big[\, \|\mathbf{z}_a - \mathbf{z}_p\|_2^2 - \|\mathbf{z}_a - \mathbf{z}_q\|_2^2 + 1 \,\big]_+. \tag{14}$$

The *anchor* objective regresses each node's normalized hop-distances to K sampled landmark nodes, $\mathbf{d}_i \in [0,1]^K$, computed by multi-source breadth-first search; *anchorcls* is its classification variant, predicting the discrete hop-distance bucket to each landmark with a cross-entropy loss.

Label-related objective.

The *hist* objective is semi-supervised. Its target is the class distribution of a node's observable k-hop neighbors, using only training labels,

$$\mathbf{p}_i = \text{normalize}\Big(\sum_{j \in \mathcal{N}_k(i) \cap V_o} w_{ij}\, \mathbf{e}_{y_j} \Big), \tag{15}$$

where $\mathbf{e}_{y_j}$ is the one-hot label of neighbor j and w_{ij} a proximity weight. The prediction is trained with a KL divergence over observable nodes that have at least one observable neighbor,

$$\mathcal{L}_{\text{hist}} = \frac{1}{|V_o|} \sum_{i \in V_o} \text{KL}\big(\mathbf{p}_i \,\|\, \text{softmax}(h_{\text{hist}}(\mathbf{z}_i))\big). \tag{16}$$

Contrastive objectives.

These do not predict a fixed target but encourage agreement between the two views $\mathbf{Z}^{(1)}, \mathbf{Z}^{(2)}$ (Section 4.4). The default is the Barlow-Twins redundancy-reduction loss of Equation (7). We additionally evaluate *InfoNCE*, which maximizes agreement between the two views of the same node against other nodes in the batch, and *prototype-InfoNCE*, which contrasts nodes against per-class prototypes formed from observable labels.

Appendix B. Hyperparameters and Design Choices

Table 9 lists the full NESS configuration used in all experiments, grouped by the two feature regimes: the large dense-embedding graph (ogbn-arxiv) and the small sparse binary bag-of-words graphs (Cora, CiteSeer, AMAP, AMAC). The architecture, self-supervised objectives, and loss weights are shared across both regimes; the settings that differ are the learning rate and the use of cluster-based batching, both dictated by graph scale rather than by tuning. These values were selected once on ogbn-arxiv (Section 6.7) and held fixed thereafter.

References

1. Hamilton, W.; Ying, Z.; Leskovec, J. Inductive representation learning on large graphs. *Advances in neural information processing systems* **2017**, *30*.

Table 9. NESS hyperparameters and design choices. The architecture and objectives are shared; only the scale-dependent settings (learning rate, batching) differ between regimes.

Setting	**ogbn-arxiv** (dense)	**Binary BoW graphs** (Cora/CiteSeer/AMAP/AMAC)
Architecture		
Prefill	Feature Propagation	Feature Propagation
Prefill iterations T	40	40
Encoder	GraphSAGE	GraphSAGE
Layers L	3	3
Hidden width h	128	128
Dropout	0.3	0.3
Encoder / decoder channels	256 / 64	256 / 64
Views and objectives		
View 1 / View 2	edge-mask / edge-mask	edge-mask / edge-mask
Edge-mask rate p	0.7	0.7
View fusion	average	average
Contrastive	Barlow-Twins ($\lambda = 0.005$)	Barlow-Twins ($\lambda = 0.005$)
SSL objectives	hist + path	hist + path
SSL neighborhood k	2	2
Path walk length ℓ	2	2
Loss weights		
w_{cls}	3.0	3.0
w_{edge}	1.0	1.0
w_{con}	0.5	0.5
w_{ssl} (hist, path)	1.0, 1.0	1.0, 1.0
Optimization		
Optimizer	Adam	Adam
Learning rate	10^{-3}	10^{-2} (10^{-3} for AMAC)
Weight decay	5×10^{-5}	5×10^{-5}
Epochs	800	800
Early stopping	val macro-F1, patience 20	val macro-F1, patience 20
Batching	cluster-based (50 parts)	full-batch

2. Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Lio, P.; Bengio, Y. Graph attention networks. *arXiv preprint arXiv:1710.10903* **2017**.
3. Chen, X.; Chen, S.; Yao, J.; Zheng, H.; Zhang, Y.; Tsang, I.W. Learning on attribute-missing graphs. *IEEE transactions on pattern analysis and machine intelligence* **2020**, *44*, 740–757.
4. Yoo, J.; Jeon, H.; Jung, J.; Kang, U. Accurate node feature estimation with structured variational graph autoencoder. Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2022, pp. 2336–2346.
5. Rossi, E.; Kenlay, H.; Gorinova, M.I.; Chamberlain, B.P.; Dong, X.; Bronstein, M.M. On the unreasonable effectiveness of feature propagation in learning on graphs with missing node features. Learning on graphs conference. PMLR, 2022, pp. 11–1.
6. Zhang, Z.; Jiang, B.; Tang, J.; Tang, J.; Luo, B. Incomplete graph learning via partial graph convolutional network. *IEEE Transactions on Artificial Intelligence* **2024**, *5*, 4315–4321.
7. Taguchi, H.; Liu, X.; Murata, T. Graph convolutional networks for graphs containing missing features. *Future Generation Computer Systems* **2021**, *117*, 155–168.
8. Peng, X.; Cheng, J.; Tang, X.; Zhang, B.; Tu, W. Multi-view graph imputation network. *Information Fusion* **2024**, *102*, 102024.
9. Zhang, X.; Li, M.; Wang, Y.; Fei, H. Amgcl: Feature imputation of attribute missing graph via self-supervised contrastive learning. *arXiv preprint arXiv:2305.03741* **2023**.
10. Yu, Y.; Li, H.; Yang, X.; Zhang, Y.; Song, J. Multi-view collaborative learning for graph attribute imputation. *International Journal of Machine Learning and Cybernetics* **2024**.
11. Hou, Z.; Liu, X.; Cen, Y.; Dong, Y.; Yang, H.; Wang, C.; Tang, J. GraphMAE: Self-Supervised Masked Graph Autoencoders. Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2022.
12. Zbontar, J.; Jing, L.; Misra, I.; LeCun, Y.; Deny, S. Barlow Twins: Self-Supervised Learning via Redundancy Reduction. International Conference on Machine Learning (ICML), 2021.
13. Hassani, K.; Khasahmadi, A.H. Contrastive Multi-View Representation Learning on Graphs. International Conference

on Machine Learning (ICML), 2020.
14. Anonymous. Neighborhood-Centric Self-Supervised Graph Models for Imputation as a Service. TODO: fill venue, 2024.
15. Liu, Y.; Ding, K.; Wang, J.; Lee, V.; Liu, H.; Pan, S. Learning strong graph neural networks with weak information. Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining, 2023, pp. 1559–1571.
16. Huo, C.; Jin, D.; Li, Y.; He, D.; Yang, Y.B.; Wu, L. T2-gnn: Graph neural networks for graphs with incomplete features and structure via teacher-student distillation. Proceedings of the AAAI Conference on Artificial Intelligence, 2023, Vol. 37, pp. 4339–4346.
17. Chen, Y.; Wu, L.; Zaki, M. Iterative deep graph learning for graph neural networks: Better and robust node embeddings. *Advances in neural information processing systems* **2020**, *33*, 19314–19326.
18. Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; Tang, J. Graph structure learning for robust graph neural networks. Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining, 2020, pp. 66–74.
19. You, J.; Ma, X.; Ding, D.Y.; Kochenderfer, M.; Leskovec, J. Handling missing data with graph representation learning. *Neural Information Processing Systems (NeurIPS* **2020**.
20. Yoon, J.; Jordon, J.; Schaar, M. Gain: Missing data imputation using generative adversarial nets. International conference on machine learning. PMLR, 2018, pp. 5689–5698.
21. Spinelli, I.; Scardapane, S.; Uncini, A. Missing data imputation with adversarially-trained graph convolutional networks. *Neural Networks* **2020**, *129*, 249–260.
22. Berg, R.v.d.; Kipf, T.N.; Welling, M. Graph convolutional matrix completion. *arXiv preprint arXiv:1706.02263* **2017**.
23. Zhang, M.; Chen, Y. Inductive matrix completion based on graph neural networks. *arXiv preprint arXiv:1904.12058* **2019**.
24. Tu, W.; Zhou, S.; Liu, X.; Liu, Y.; Cai, Z.; Zhu, E.; Zhang, C.; Cheng, J. Initializing Then Refining: A Simple Graph Attribute Imputation Network. IJCAI, 2022, pp. 3494–3500.
25. Tu, W.; Xiao, B.; Liu, X.; Zhou, S.; Cai, Z.; Cheng, J. Revisiting initializing then refining: An incomplete and missing graph imputation network. *IEEE Transactions on Neural Networks and Learning Systems* **2024**, *36*, 3244–3257.
26. Jin, D.; Wang, R.; Wang, T.; He, D.; Ding, W.; Huang, Y.; Wang, L.; Pedrycz, W. Amer: A new attribute-missing network embedding approach. *IEEE Transactions on Cybernetics* **2022**, *53*, 4306–4319.
27. Um, D.; Park, J.; Park, S.; Choi, J.Y. Confidence-based feature imputation for graphs with partially known features. *arXiv preprint arXiv:2305.16618* **2023**.
28. Veličković, P.; Fedus, W.; Hamilton, W.L.; Liò, P.; Bengio, Y.; Hjelm, R.D. Deep Graph Infomax. International Conference on Learning Representations (ICLR), 2019.
29. Peng, Z.; Huang, W.; Luo, M.; Zheng, Q.; Rong, Y.; Xu, T.; Huang, J. Graph Representation Learning via Graphical Mutual Information Maximization. Proceedings of The Web Conference (WWW), 2020.
30. Zhu, Y.; Xu, Y.; Yu, F.; Liu, Q.; Wu, S.; Wang, L. Deep Graph Contrastive Representation Learning. *arXiv preprint arXiv:2006.04131* **2020**.
31. Zhu, Y.; Xu, Y.; Yu, F.; Liu, Q.; Wu, S.; Wang, L. Graph Contrastive Learning with Adaptive Augmentation. Proceedings of The Web Conference (WWW), 2021.
32. You, Y.; Chen, T.; Sui, Y.; Chen, T.; Wang, Z.; Shen, Y. Graph Contrastive Learning with Augmentations. Advances in Neural Information Processing Systems (NeurIPS), 2020.
33. Lee, N.; Lee, J.; Park, C. Augmentation-Free Self-Supervised Learning on Graphs. Proceedings of the AAAI Conference on Artificial Intelligence, 2022.
34. Gong, X.; Yang, C.; Shi, C. MA-GCL: Model Augmentation Tricks for Graph Contrastive Learning. Proceedings of the AAAI Conference on Artificial Intelligence, 2023.
35. Peng, X.; others. Dual Contrastive Learning Network for Graph Clustering. *IEEE Transactions on Neural Networks and Learning Systems* **2023**.
36. Li, J.; Wu, R.; Sun, W.; Chen, L.; Tian, S.; Zhu, L.; Meng, C.; Zheng, Z.; Wang, W. What's Behind the Mask: Understanding Masked Graph Modeling for Graph Autoencoders. Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2023.
37. Tu, W.; Liao, Q.; Zhou, S.; Peng, X.; Ma, C.; Liu, Z.; Liu, X.; Cai, Z. RARE: Robust Masked Graph Autoencoder. *arXiv preprint arXiv:2304.01507* **2023**.
38. Xia, R.; Zhang, C.; Li, A.; Liu, X.; Yang, B. Attribute imputation autoencoders for attribute-missing graphs. *Knowledge-Based Systems* **2024**, *291*, 111583.
39. He, D.; others. Contrastive Attribute Completion for Heterogeneous Graphs. *IEEE Transactions on Neural Networks and Learning Systems* **2022**.
40. Moshiri, E.; Wu, J.; Newton, M.A.H.; Li, M.; Asgari, P.; Beheshti, A. Neighborhood-Centric Self-Supervised Graph Models for Imputation as a Service. IEEE International Conference on Web Services (ICWS), 2026, pp. 803–812.
41. Lin, Y.; Gou, Y.; Liu, Z.; Li, B.; Lv, J.; Peng, X. COMPLETER: Incomplete Multi-view Clustering via Contrastive Prediction. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021.

42. Pan, E.; Kang, Z. Multi-view Contrastive Graph Clustering. Advances in Neural Information Processing Systems (NeurIPS), 2021.
43. Lei, C.; Fu, S.; Wang, Y.; Qiu, W.; Hu, Y.; Peng, Q.; You, X. Self-Supervised Guided Hypergraph Feature Propagation for Semi-Supervised Classification with Missing Node Features. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2023.
44. Chiang, W.L.; Liu, X.; Si, S.; Li, Y.; Bengio, S.; Hsieh, C.J. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining, 2019, pp. 257–266.
45. Yang, Z.; Cohen, W.; Salakhudinov, R. Revisiting semi-supervised learning with graph embeddings. International conference on machine learning. PMLR, 2016, pp. 40–48.
46. Shchur, O.; Mumme, M.; Bojchevski, A.; Günnemann, S. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* **2018**.
47. Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; Leskovec, J. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* **2020**, *33*, 22118–22133.